

Neighbour-Link

Neighbour-Link (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Social Media · **Assessed:** 2026-09-19 · **Status:** awaiting instructor approval

Proposal (as submitted): An application designed to revive lost community spirit by connecting neighbours who share common interests. Users are matched by location (postal code) and interest, can discover a hobbies feed, create neighbourhood events, and earn reputation and rewards (points redeemable for shop discounts). A C++ service continuously tracks and manages nearby users and events; a "Dynamic Hobby Creator" lets users define new hobbies. Named building blocks: location-based matching (postal code), database management, interest-matching algorithms, and a real-time notification system. Four major Design features: hobbies-discovery feed, neighbourhood event creation, interest matching, and reputation & reward.

Adjustments required for approval:

- The **interest + postal-code proximity matching must be the team's own load-bearing C++ service** (a benchmarked ranking/matching engine with weighted scoring, not a Firebase query or a plain database `where` filter). The proposal already commits to "a C++ service that continuously tracks and manages nearby users and events" — hold the design to that; Firebase stays the store/sync layer, never the matcher (category guard-rail: Firebase alone must never be the backend).
- **Safety-by-design, because the product connects strangers to meet offline:** verified accounts, report + block, per-user visibility controls, and a **moderation / report-review dashboard** (this is the custom tool) — plus in-app public-place guidance for event meetups. Treat this as a graded surface, not an afterthought.
- **Reputation and rewards computed server-side** (anti-gaming) and the "shop discount" rewards must stay **in-app points** — a points ledger plus

redeemable codes, **not real payments or e-commerce**. Real money would push the project into Fintech and out of scope for this trimester.

References:

- [Nextdoor](#) — the neighbourhood network (verified neighbours, local events, shared-interest connections)
- [Meetup](#) — find local groups & events near you around shared interests
- [Bumble For Friends](#) — match nearby people by shared interests, with identity verification and IRL group plans

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Neighbour-Link
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	♦
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile + Web browser on mobile — location-based matching and push notifications are mobile-first (GPS/postal geocoding + FCM push favour a native build), with a mobile-web front-end for reach; the Native PC build is the development/iteration and tools vehicle (device: Pixel 9a). **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: Hobbies-discovery feed, nearby-neighbours / event proximity view, event-creation and hobby-creator screens, reputation/points display and

notification centre (OpenGL ES for the custom feed cards, proximity map and reputation charts). **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES); Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: The C++ matching/tracking service (interest + postal-code proximity scoring, incremental re-ranking, nearby-users/events index), notification dispatch, reputation/points ledger and moderation/report queue.

Languages: C++.

Backend scope: Local now, server-ready (T2) — the Linux server seam is set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the load-bearing piece is a **performant C++ service** that continuously maintains the nearby-users/events index and re-scores interest + postal-code proximity matches as users, hobbies and events change — weighted scoring and incremental re-ranking in C++, not a database query. Real-time notification dispatch runs off the same service.
- *Data-driven:* hobbies (including user-authored ones from the Dynamic Hobby Creator), events, interest tags, matching weights, reputation rules and reward tiers are all hot-reloadable JSON — no hard-coded hobbies, event types or scoring constants.
- *2D:* custom-rendered feed cards, a nearby-neighbours / event proximity view, and reputation/engagement charts (custom-rendered, no chart libs) — the domain-adapted 2D surface for a Social Media app. *3D:* not needed.
Networking: real-time notifications (new nearby match, event invite, reputation change) and presence of nearby users/events via push + an in-app event bus.
- *Custom tool:* the **Dynamic Hobby Creator** (author/curate hobby definitions and interest tags) plus a **moderation / report-review dashboard** (PC-side authoring/ops) — review the report queue, action blocks, and tune matching weights and reward tiers.
- *External DB:* Firebase stores users, hobbies, events, reputation ledger and reports. *Auth »:* verified accounts are safety-critical here (identity/postal verification, report/block tied to identity). *Analytics »:* optional engagement funnels — which hobbies/events convert to real-life meetups.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window and input with a fixed-timestep loop driving the nearby-users/events tick, match-rescoring cadence and notification dispatch; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the M1G9 matching engine and M1G12 notifications run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ entity/registry with users, hobbies, interest tags, events, matching weights and reputation/reward rules defined in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & update hooks).
- **Feeds into:** M1G7 (hobby/event content models), M1G9 (scoring reads this model), M2G2 (the Hobby Creator edits this data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for the match index and notification queue, Tracy profiling, Catch2 stress tests re-ranking a synthetic neighbourhood (tens of thousands of users/events across many postal codes at once), GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the frame/latency budget M2G1 validates for web/mobile.

- **G7 Data-Driven Content Models** — hobbies, events and reputation rules as JSON schemas: the data backbone the Dynamic Hobby Creator authors.
Tech & dependencies

- **Tech:** Versioned JSON schemas for hobbies/interest tags, event types and reputation/reward tiers, with schema validation and hot-reload.
- **Prerequisites:** G2 (data-driven architecture).
- **Feeds into:** M1G9 (scoring features), M2G2 (Hobby Creator), M2G8 (recommendation features).

- **G9 Matching / Ranking Engine — Foundation** — the load-bearing C++ service: score neighbours and events by shared interest + postal-code proximity.

Tech & dependencies

- **Tech:** Priority queues and weighted scoring functions (interest-tag overlap + postal-code distance), incremental re-ranking as users/events change, a maintained nearby index (custom C++) — not a Firebase query.
- **Prerequisites:** G2 (user/hobby/event model), G1 (tick).
- **Feeds into:** M2G8 (recommendations II), the discovery feed and match notifications.

- **G12 Notification System — Foundation** — real-time alerts for new nearby matches, event invites and reputation changes.

Tech & dependencies

- **Tech:** Firebase Cloud Messaging (push) plus an in-app event bus and a web service worker; notification schema and delivery tracking.
- **Prerequisites:** G1 (loop/event bus); Firebase integration seam.
- **Feeds into:** M2G12-class digests and the M2G7 push/identity completion.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of match re-ranking spikes, ASan on MSVC, memory pools for the nearby index, and Emscripten WASM validation for the mobile-web target plus on-device Pixel 9a checks; Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where

realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The web/mobile deliverable and headroom for larger neighbourhoods.

- **G2 ★ Content Editor — Core** — the Dynamic Hobby Creator plus the moderation/report-review dashboard: author hobbies/interest tags, tune matching weights and reward tiers, action reports.

Tech & dependencies

- **Tech:** ImGui editor authoring hobby/tag definitions, matching-weight and reward-tier tables as JSON, with round-trip serialization and an undo/redo command stack; a report-queue review pane.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL renderer (live feed/proximity preview).
- **Prerequisites:** G2 (hobby/event/reputation data), M1G7 (content models).
- **Feeds into:** M2G8 (weights it tunes), M2G16 (safety actions applied through it).

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the native build plus the Emscripten web build artifact via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every Neighbour-Link goal.

- **G8 Ranking / Matching Engine II** — recommendations: rank neighbour and event suggestions by interest + proximity + reputation.

Tech & dependencies

- **Tech:** Scoring models (weighted features, cosine similarity on interest-tag vectors), candidate generation from the nearby index, and evaluation metrics; reputation as a scoring feature.
- **Prerequisites:** M1G9 (matching foundation), M1G7 (content/feature models).

- **Feeds into:** The discovery feed, "neighbours you may know" and event/meetup suggestions.
- **G7 Firebase — Auth, Push & Analytics** — verified accounts, push delivery and the reputation/reward ledger sync.

Tech & dependencies

 - **Tech:** Firebase Auth (verified accounts — safety-critical), Firestore/Realtime DB for hobbies/events/reputation, Cloud Messaging for push and Analytics — REST API from C++, JS SDK on the web front-end.
 - **Prerequisites:** G2 (data model to sync), M1G12 (notification foundation).
 - **Feeds into:** M2G16 (identity-backed safety controls) and the points/reputation ledger.
- **G16 Privacy & Safety Controls** — blocking, visibility rules and report actions: the safety layer for connecting strangers offline.

Tech & dependencies

 - **Tech:** Block lists, per-user visibility flags, report-to-action flow and Firebase security rules; ties to the moderation dashboard.
 - **Prerequisites:** M2G7 (auth/identity), M2G2 (moderation dashboard).
 - **Feeds into:** Safe discovery, safe event meetups and trustworthy reputation.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 1 Art (8) — the four Design features (feed, event creation, interest matching, reputation & reward) are a strong single-designer load, and the art surface (feed cards, hobby/interest iconography, reputation badges & reward tiers, event imagery) is a utility social-app surface one artist can cover (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). Add a 2nd artist only if a richer illustrated hobby/badge set becomes a selling point.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the discover → match → create/join event → earn reputation loop, plus the Dynamic Hobby Creator and the in-app points/reward economy — anti-gaming)
- M1D2 Prototype (clickable flow proving the interest-match → event-join path feels worthwhile)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (matching weights and reward tiers tuned via the team's JSON, no code changes)

- **UI/UX M1** (graded by the UI/UX instructor):

- M1U1 ★ UI Standards & Wireframe Baseline — screen-flow map and readability standards for the feed, matching, event-creation and reputation screens.
- M1U2 UX Flows & Wireframes — the discover → match → join-event and hobby-creation journeys.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the required Social Media screens (feed, profile, event, notifications, report/block) navigable unaided.
- M2U2 ★ UX Validation & Final Polished UI — touch-first layouts, notification affordances and proximity view validated on the Pixel 9a / mobile-web target.

- **Art M1:** 1 artist (utility social-app surface)
 - M1A1 ★ (mockups: hobbies-discovery feed + event-creation screen)
 - M1A4 UI Art Concepts & Typography (feed cards, hobby/interest icon set, reputation/reward visual language)
 - M1A6 Early Asset Production (first production-ready icon & badge set)
- **Art M2:**
 - M2A1 ★ (polished feed, event and reputation screens)
 - M2A4 UI Art — States & Feedback (match found, event full, reputation up/down, reward unlocked)
 - M2A5 Illustration & Iconography — Full Set (complete hobby/interest icons, reward-tier badges, empty/onboarding states)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial UI/notification SFX and light ambient BGM from the week-5 needs list, present & working in the prototype (match found, event invite, reputation change), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback (overlapping notification/UI SFX).
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Social Media floor (subtle UI/notification SFX + optional light ambient BGM in the showcase build; audio is not load-bearing here).
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing for stacked notifications).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase, prototype demo, team post-mortem (wk14)